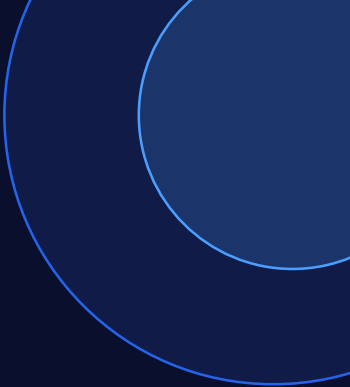




ARCHITECTURE SERIES

From Monolith to **Microservices**

A journey through modern software architecture

What We'll Cover

01

Monolithic Architecture

What it is, how it works, and why it was the standard

03

Microservices Introduction

The concept, principles, and core ideas

02

Challenges of Monoliths

Pain points that emerge as systems grow

04

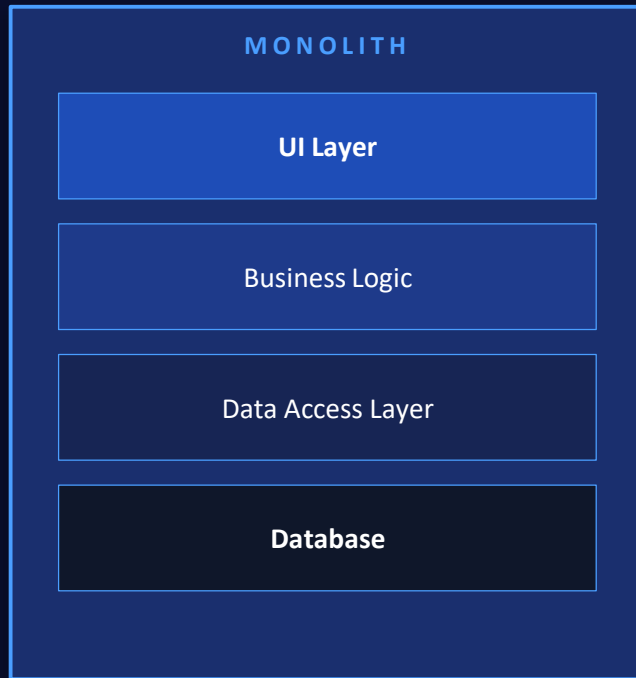
Key Differences

Comparing both approaches side by side

Everything in One Place

A monolithic application is built as a single, unified unit where all components — UI, business logic, and data access — are tightly coupled and deployed together.

- Single deployable artifact (one big app)
- All features share the same process & memory
- One database for the entire application
- Any change requires a full redeployment



The Growing Pains



Scalability Limits

You must scale the entire app even if only one feature needs more power



Tight Coupling

Changing one module risks breaking everything else



Slow Deployments

A small bug fix requires redeploying the entire application



Team Bottlenecks

Large teams struggle to work independently on the same codebase



Tech Lock-in

The whole app must use the same tech stack and language

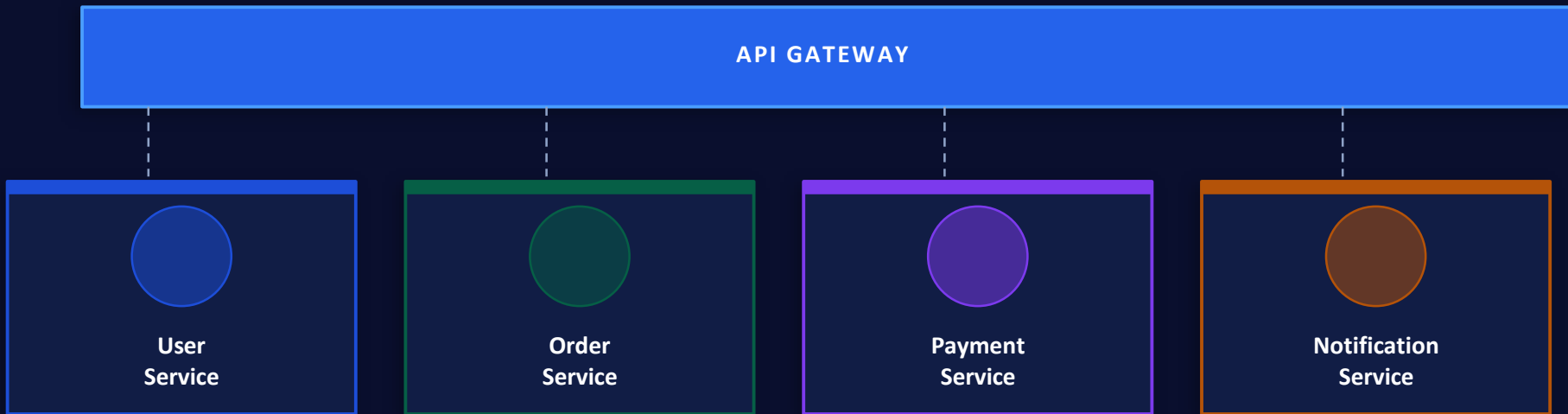


Single Point of Failure

One crash can bring down the entire system

Small, Independent, Powerful

Microservices is an architectural approach where an application is built as a collection of small, loosely coupled services — each responsible for a specific business capability, deployable and scalable independently.



Monolith vs Microservices

	📄 MONOLITH	⚡ MICROSERVICES
Deployment	All-or-nothing	Each service independently
Scalability	Scale everything	Scale what's needed
Codebase	One large codebase	Many small repositories
Tech Stack	Single tech stack	Best tool per service
Fault Isolation	One bug = full crash	Failures are contained
Team Structure	Shared ownership	Small autonomous teams

More details coming in the next sessions →